UNIVERSIDADE FEDERAL DA PARAÍBA

Centro de Informática - Ciência da Computação

PROJETO DE ENGENHARIA DE SISTEMAS DISTRIBUÍDOS

DOCUMENTO ARQUITETURAL

Equipe: Arthur Vieira

Augusto Miguel

Carlos Eduardo

Igor Wanderley

Joás Gomes

Kalil Teotonio

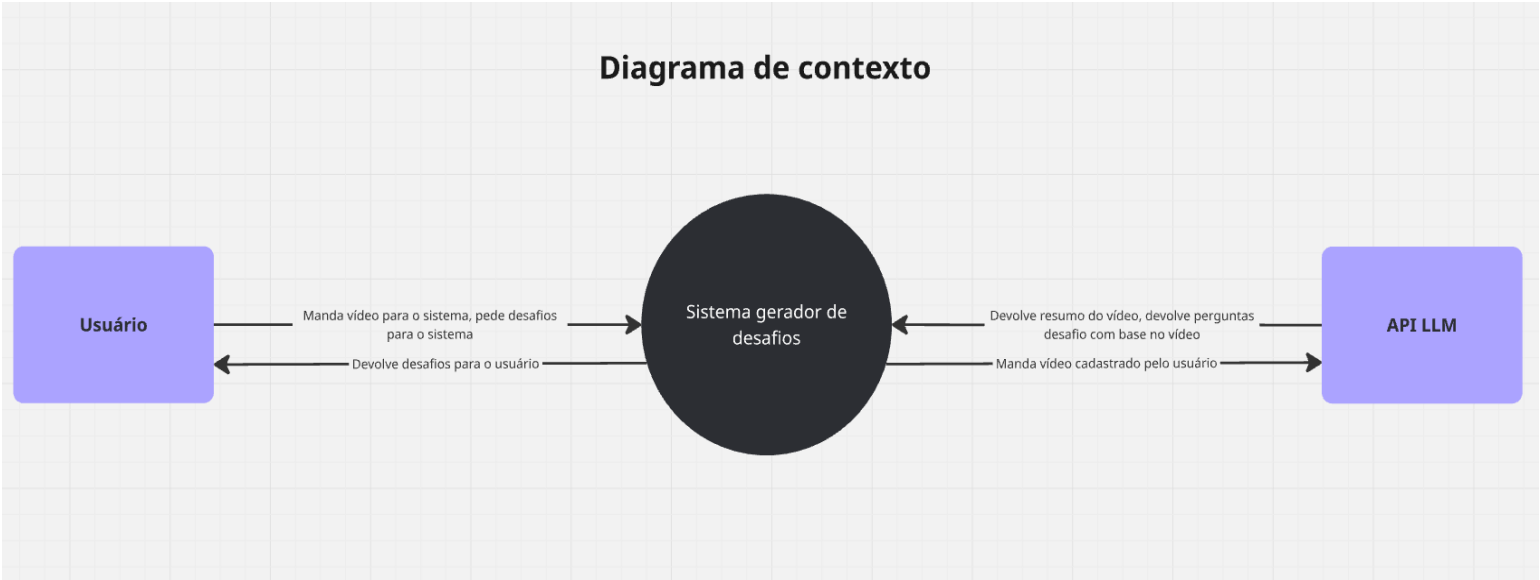
João Pessoa - PB

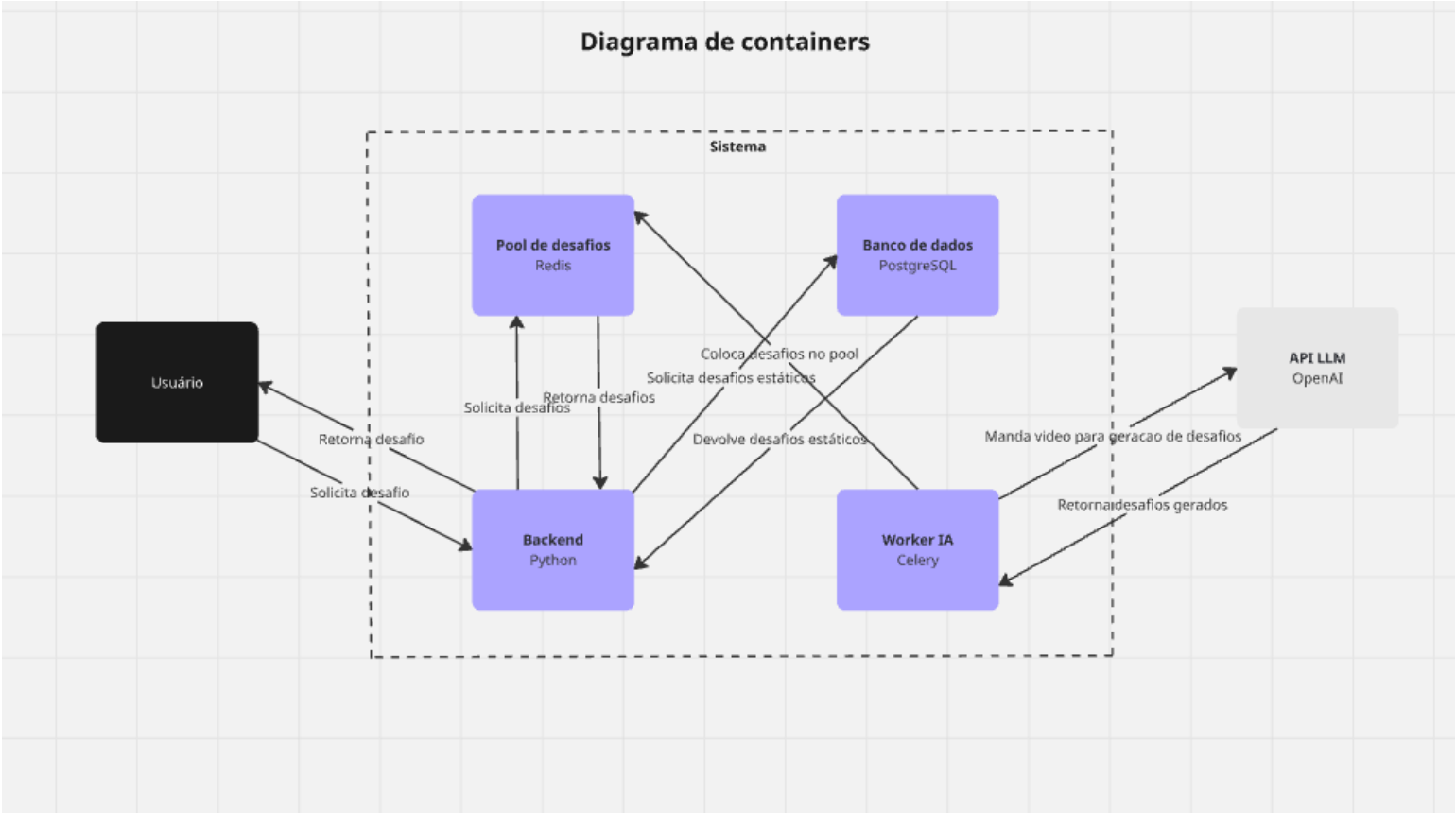
2026

Sumário

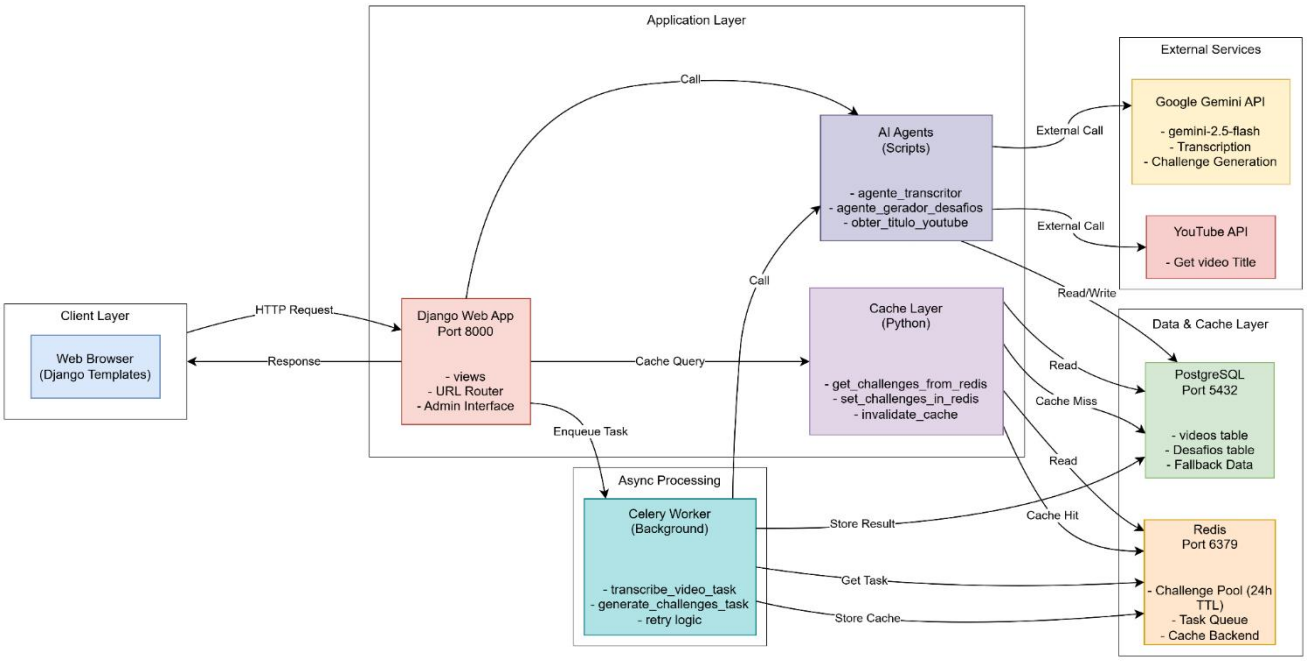
1. Diagramas de contexto e containers 3
2. Diagrama C4 4
3. ADRs iniciais 4
4. Definição de stack tecnológico e justificativas 6
5. Identificação dos padrões arquiteturais 6
6. Divisão de responsabilidades entre os integrantes 7

1. Diagramas de contexto e containers





2. Diagrama C4



3. ADRs iniciais

ADR 01: Adoção de Processamento Assíncrono com Redis para o Pool.

Contexto: A geração de desafios pela IA (LLM) é um processo naturalmente lento e sujeito a oscilações de rede. Se o motor do jogo esperasse essa geração em tempo real, a experiência do usuário travaria.

Decisão: Optamos por implementar o processamento ASYNC acoplado ao padrão de Queues utilizando o Redis. O Redis atuará simultaneamente como corretor de mensagens (broker) para o worker e como cache em memória rápida para armazenar o pool de desafios gerados.

Consequências:

Positivo: O motor do jogo consegue consumir as perguntas do pool instantaneamente, pois o fluxo crítico é isolado da latência da IA.

Negativo (Trade-off): Adiciona complexidade à infraestrutura do projeto, exigindo o monitoramento de um serviço extra (Redis) e o gerenciamento de workers em background (Celery no Django).

ADR 02: Implementação de Circuit Breaker para a API da LLM

Contexto: O preenchimento do pool depende de uma API externa de IA (ex: GPT-4o-mini). APIs externas são pontos únicos de falha, podendo sofrer timeouts ou indisponibilidade total, o que comprometeria o worker.

Decisão: Adotamos o padrão Circuit Breaker para encapsular todas as requisições feitas à IA. Se a taxa de erros ou o tempo de resposta da OpenAI ultrapassar um limite tolerável, o disjuntor "abre", interrompendo novas requisições. Nesse momento, o sistema faz o fallback automático, consumindo perguntas de um banco de dados estático (PostgreSQL).

Consequências:

Positivo: Garante a resiliência contínua do sistema, evitando travamentos sistêmicos por lentidão de terceiros.

Negativo (Trade-off): Quando o disjuntor está aberto, o sistema perde sua característica principal: as perguntas servidas aos jogadores não terão contexto direto com os vídeos, diminuindo o valor temporário da solução.

ADR 03: Usar Google Gemini 2.5 Flash ao invés de OpenAI GPT-4

Contexto: O projeto precisa processar transcrição de áudio de vídeos do YouTube em tempo real; desafios incluem latência crítica (~5-10s), qualidade adequada para educação e time pequeno; avaliação inicial entre GPT-4, Gemini e Whisper.

Decisão: Usar Google Gemini 2.5 Flash como modelo primário para transcrição e geração de desafios, devido ao melhor equilíbrio entre latência, custo e qualidade.

Consequências:

Positivo: Performance otimizada (latência reduzida 47%), qualidade adequada para educação, confiabilidade estruturada (JSON mode nativo) e escalabilidade/custo reduzido.

Negativo (Trade-off): Leve redução na acurácia semântica, acoplamento a Google (vendor lock-in) e maturidade menor da documentação/comunidade.

4. Definição de stack tecnológico e justificativas

Linguagem Base e API: Python

Justificativa: Python é a linguagem padrão da indústria para integrações com Inteligência Artificial e processamento de dados. Seu ecossistema rico e frameworks web robustos permitem o desenvolvimento ágil da API síncrona que atenderá o motor do jogo, além de facilitar a comunicação fluida com a API da LLM escolhida para a geração de desafios.

Processamento Assíncrono e Workers: Celery

Justificativa: O núcleo desta POC exige que a geração de desafios seja desacoplada do fluxo crítico do usuário. O Celery atua como um gerenciador de filas de tarefas distribuídas, permitindo que a geração baseada no conteúdo do vídeo ocorra de forma assíncrona em *background*. Isso garante que a latência da requisição à IA não impacte a experiência em tempo real do jogador.

Armazenamento em Memória e Mensageria (Pool): Redis

Justificativa: O Redis foi escolhido por sua dupla função essencial nesta arquitetura. Primeiro, atua como o *Message Broker* (corretor de mensagens) exigido pelo Celery para gerenciar a fila de tarefas assíncronas. Segundo, opera como o *Cache/Pool* em memória principal, armazenando os desafios recém-gerados pela IA para garantir que a API os consuma e entregue ao motor do jogo com latência de milissegundos.

Armazenamento Persistente e Fallback: PostgreSQL

Justificativa: Um banco de dados relacional maduro e altamente confiável para garantir a persistência segura das informações de longo prazo. O PostgreSQL assume um papel crítico de resiliência: ele armazenará o banco estático de perguntas pré-aprovadas, atuando como o sistema de *fallback* imediato caso o *pool* do Redis esvazie ou a API da IA fique indisponível.

5. Identificação dos padrões arquiteturais

SYNC vs ASYNC & Queues (Filas): O motor do jogo pede o desafio para a sua API e precisa de uma resposta na mesma hora (Fluxo Síncrono). No entanto, a geração do desafio pela IA demora, então ela é desacoplada do fluxo crítico operando como um pool assíncrono. O worker (ex: Celery) usa filas (Queues) no Redis para processar essas gerações em segundo plano sem travar o jogo.

Circuit Breaker (Disjuntor): Se a API da LLM cair ou ficar muito lenta, o Circuit Breaker é acionado. Imediatamente, seu sistema para de tentar chamar a IA e aciona o fallback para o banco estático (Supabase). Isso atende exatamente ao requisito de testar a operação com a IA indisponível

Cache-Aside Pattern: A sua API atua consumindo o pool em tempo real. Ela tenta pegar o desafio no Redis (Cache). Se o Redis estiver vazio (esgotou o pool), ela vai buscar as perguntas pré-cadastradas no banco estático.

6. Divisão de responsabilidades entre os integrantes

- Arthur Vieira – Desenvolvedor
- Augusto Miguel - UI/UX
- Carlos Eduardo - Testes
- Igor Wanderley - Desenvolvedor
- Joás Gomes - Documentação
- Kalil Teotônio -Testes